1. User and permission management

Command	Purpose
`whoami`	Displays the current logged-in username.
`id`	Displays user ID (UID), group ID (GID), and group memberships.
`groups`	Shows the groups to which the current user belongs.
`sudo command`	Executes a command with administrative privileges.
`su username`	Switches to another user.
`su - username`	Switches user and loads their login environment.
`sudo -i`	Opens an interactive root shell, if permitted.
`exit`	Exits the current shell or user session.
`passwd`	Changes the current user's password.

```bash
whoami
```

Output:

```
akankshya
```

```bash
groups
```

Output:

```
akankshya adm cdrom sudo dip plugdev users
```

Important concepts:

- Normal user: Usually represented by `$`.
- Root user: Usually represented by `#`.
- `sudo`: Temporarily executes a command with administrative privileges.
- Root user: Has administrative control over the Linux system.

2. Package management – APT

APT stands for Advanced Package Tool.

It is used to install, update and remove software in Ubuntu.

Command	Purpose
`sudo apt update`	Updates the package lists.
`sudo apt upgrade`	Upgrades installed packages.
`sudo apt install package-name`	Installs a package.
`sudo apt remove package-name`	Removes an installed package.
`sudo apt purge package-name`	Removes a package and its system configuration files.
`sudo apt autoremove`	Removes automatically installed packages no longer required.
`apt list --installed`	Lists installed packages.

Examples:

```bash
sudo apt update
```

```bash
sudo apt install htop
```

Bash

```
sudo apt install nginx
```

Remember:

- `update` = Refresh package information.
- `upgrade` = Upgrade installed software.
- `install` = Install new software.

3. System monitoring commands

These commands help you monitor your Linux system.

Command	Purpose
`htop`	Interactive system and process monitor.
`top`	Displays running processes and resource usage.
`free -h`	Displays RAM and swap memory usage.
`df -h`	Displays disk space usage.
`ps aux`	Displays running processes.
`clear`	Clears the terminal screen.

Examples:

Bash

```
top
```

Exit using `q`.

Bash

```
free -h
```

Bash

```
df -h
```

Bash

```
ps aux
```

Note: `htop` may need to be installed first:

Bash

```
sudo apt install htop
```

`cd folder-name`	Enters a folder.
`cd ..`	Moves to the parent directory.
`cd ~`	Goes to the home directory.
`cd /`	Goes to the root directory.
`mkdir folder-name`	Creates a directory.
`mkdir -p a/b/c`	Creates nested directories.
`touch file.txt`	Creates an empty file if it doesn't exist.
`cat file.txt`	Displays file contents.
`cp source destination`	Copies a file.
`mv source destination`	Moves or renames a file.
`find path -name "filename"`	Searches for a file or directory.
`find path -type f`	Finds regular files.
`find path -type f -name "*.txt"`	Finds text files.

Important systemctl commands

Command	Purpose
`systemctl status nginx`	Checks Nginx service status.
`systemctl is-active nginx`	Checks whether Nginx is currently running.
`systemctl is-enabled nginx`	Checks whether Nginx starts automatically at boot.
`sudo systemctl start nginx`	Starts Nginx.
`sudo systemctl stop nginx`	Stops Nginx.
`sudo systemctl restart nginx`	Restarts Nginx.
`sudo systemctl reload nginx`	Reloads Nginx configuration.
`sudo systemctl enable nginx`	Enables automatic startup.
`sudo systemctl disable nginx`	Disables automatic startup.

Understanding your actual output

You received:

```
Plain text
active
```

This means Nginx is running.

You received:

```
Plain text
enabled
```

This means Nginx is configured to start automatically during boot.

You also saw:

Plain text

```
nginx: master process
nginx: worker process
```

- Master process: Manages the Nginx server and worker processes.
- Worker processes: Handle incoming client requests.

Commands to check users

1. `whoami`

Shows the currently logged-in username.

```bash
whoami
```

2. `id`

Displays UID, GID and group memberships.

```bash
id
```

Example output:

```
uid=1000(akankshya) gid=1000(akankshya) groups=1000(akankshya),27(sudo)
```

Understanding this:

- `uid=1000` : Your user ID.
- `gid=1000` : Your primary group ID.
- `groups` : Groups you belong to.

3. `who`

Shows users currently logged into the system.

```bash
who
```

4. `groups`

Shows the groups of the current user.

```bash
groups
```

---

2. Understanding groups in Linux

A group is simply a collection of users.

Imagine you are working in a company.



Instead of giving permission to each person individually, we can give permission to the entire group.

Commands for groups

Command	Purpose
`groups`	Shows your groups.
`id`	Shows user and group IDs.
`getent group`	Lists groups from configured group databases.
`sudo groupadd developers`	Creates a group.
`sudo groupdel developers`	Deletes a group.
`sudo usermod -aG developers username`	Adds a user to a supplementary group.
`sudo gpasswd -d username developers`	Removes a user from a group.

3. Understanding Linux permissions

This is the most important part.

Linux controls who can read, modify or execute a file.

There are three permission categories:

Symbol	Meaning
`r`	Read
`w`	Write
`x`	Execute

And three categories of users:

Symbol	Meaning
`u`	User (owner)
`g`	Group
`o`	Others

Let's understand with an example.

```bash
-rwxr-xr-- 1 akankshya developers 100 Nov 4 notes.txt
```

Breaking down the permission string

-rwxr-xr--

Owner	Group	Others
rwx	r-x	r--
Read, write, execute	Read, execute	Read only

The first `-` means this is a regular file.

Remember:

- Owner = Person who owns the file.
- Group = Members of the associated group.
- Others = Everyone else.

What does each permission mean?

Permission	File	Directory
Read (`r`)	View contents	List directory entries
Write (`w`)	Modify contents	Create, delete or rename entries (subject to other restrictions)
Execute (`x`)	Run executable file	Enter/traverse directory

4. How to check permissions

Use:

```bash
ls -l
```

Example:

```
-rw-r--r-- 1 akankshya akankshya 25 Nov 4 notes.txt
```

Interpretation:

- `-` = Regular file.
- `rw-` = Owner can read and write.
- `r--` = Group can only read.
- `r--` = Others can only read.
- First `akankshya` = Owner.
- Second `akankshya` = Group.

To check permissions of a specific file:

```bash
ls -l notes.txt
```

5. chmod – Changing permissions

`chmod` stands for Change Mode.

It changes file or directory permissions.

There are two ways to use it.

A. Symbolic method

Command	Meaning
`chmod u+x file.txt`	Add execute permission to owner.
`chmod g+w file.txt`	Add write permission to group.
`chmod o-r file.txt`	Remove read permission from others.
`chmod a+x file.txt`	Add execute permission for everyone.
`chmod u-w file.txt`	Remove write permission from owner.

Remember:

- `+` = Add permission.
- `-` = Remove permission.
- `=` = Set exact permission.

Example:

```bash
chmod u+x notes.txt
```

Now the owner gets execute permission.

6. chown – Changing ownership

`chown` stands for Change Owner.

Suppose `notes.txt` belongs to Akankshya, but you want Rahul to own it.

```bash
sudo chown rahul notes.txt
```

To change both owner and group:

```bash
sudo chown rahul:developers notes.txt
```

To change ownership recursively for a directory and its contents:

```bash
sudo chown -R rahul:developers my-folder
```

7. chgrp – Changing group ownership

`chgrp` stands for Change Group.

Example:

```bash
sudo chgrp developers notes.txt
```

This changes the group associated with the file to `developers`.

---

8. Important commands – Quick revision

Command	Purpose
`whoami`	Current username
`id`	UID, GID and groups
`groups`	User's groups
`sudo groupadd dev`	Create group
`sudo groupdel dev`	Delete group
`sudo usermod -aG dev username`	Add user to group
`ls -l`	Check permissions
`chmod 755 file`	Change permissions
`chown user file`	Change owner
`chgrp group file`	Change group
`umask`	View default permission mask